

ASSIGNMENT 8 — REST API & LOCAL CACHING IN FLUTTER

[Github Link](#)

1. INTRODUCTION

The objective of this assignment was to develop a Flutter application that communicates with a REST API, processes JSON data, displays it dynamically, and uses local caching when network connectivity is unavailable.

For this assignment, I developed PulseAPI, a modern API-driven dashboard that retrieves meaningful post data from a REST API and presents it through a responsive user interface.

The application follows the data flow:

REST API → JSON Response → Dart Model → UI

When fresh data is successfully retrieved, it is also stored locally. If the API cannot be reached later, the application can display previously stored data from the local cache.

This demonstrates an important real-world concept used in modern mobile applications: network data with offline/cache fallback.

2. OBJECTIVES

The main objectives of this assignment were:

- To understand REST API communication in Flutter.
- To make HTTP GET requests using the http package.
- To understand JSON responses and convert them into Dart objects.
- To create a reusable data model for API information.
- To display API data dynamically in Flutter.
- To implement loading, success, empty and error states.
- To store API results locally using SharedPreferences.
- To retrieve cached data when the API is unavailable.
- To implement refresh functionality.
- To create a responsive and professional API dashboard.

3. TECHNOLOGIES USED

Flutter & Dart:

Flutter was used to build the application interface, while Dart was used for the application logic, API communication, asynchronous operations and data processing.

HTTP:

The http package is used to communicate with the REST API through HTTP GET requests.

SharedPreferences:

The shared_preferences package is used for local storage. Successfully retrieved API data is stored locally so that it can be reused when the network is unavailable.

REST API:

A public REST API is used to retrieve post information such as:

- Post ID
- User ID
- Title
- Body

4. PROJECT STRUCTURE

The application is organized into separate layers:

```
lib/
├── main.dart
├── models/
│   └── post.dart
├── services/
│   ├── api_service.dart
│   └── cache_service.dart
├── screens/
│   └── home_screen.dart
└── utils/
    └── app_theme.dart
```

Model:

The post.dart file represents the structure of the API data and converts JSON objects into Dart Post objects.

API Service:

The api_service.dart file is responsible for communicating with the REST API, making HTTP requests, validating responses and converting JSON data into Post objects.

Cache Service:

The cache_service.dart file manages local storage using SharedPreferences. It saves and retrieves posts and also stores information about when the cache was updated.

Home Screen:

The `home_screen.dart` file handles the user interface and displays the retrieved information through cards, statistics, status indicators and responsive layouts.

5. REST API IMPLEMENTATION

The application communicates with the REST API using an asynchronous HTTP GET request.

The process is:

User opens application



ApiService



HTTP GET Request



REST API



JSON Response



JSON Parsing



List<Post>



Display in UI

Since network requests take time, asynchronous programming using `Future`, `async` and `await` is used.

The application also checks the HTTP response and handles unsuccessful responses instead of allowing them to crash the application.

6. JSON PARSING AND DATA MODEL

The API returns information in JSON format. Instead of directly displaying raw JSON, the application converts the JSON objects into structured Dart Post objects.

The process is:

JSON Data



`Post.fromJson()`



Post Object



Flutter UI

The Post model contains important properties such as:

- userId
- id
- title
- body

The model also provides additional presentation-related information such as author information, category, word count and estimated reading time.

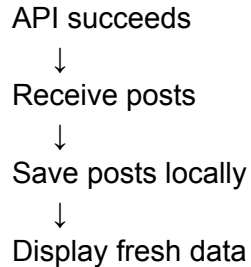
This approach makes the application easier to maintain and keeps the data structure organized.

7. LOCAL CACHING

Local caching is one of the main features of this assignment.

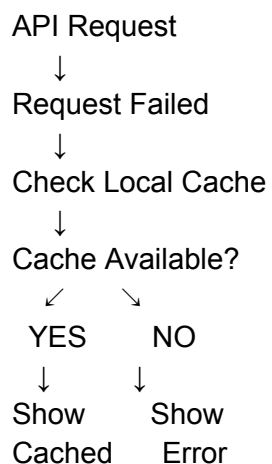
Whenever fresh data is successfully received from the API, the application stores the data locally using SharedPreferences.

The process is:



If the API request fails, the application checks whether previously stored data is available.

The fallback process is:



Data State

This improves application reliability because previously loaded information can still be displayed even when the network is temporarily unavailable.

The cache service also stores a timestamp, which allows the application to determine how old the cached data is.

8. USER INTERFACE AND FEATURES

PulseAPI was designed as a modern API dashboard instead of simply displaying raw JSON information.

Main features include:

- Modern dashboard interface
- Responsive desktop, tablet and mobile layouts
- API statistics
- Latest post cards
- User information
- Loading state
- Error state
- Empty-data handling
- Refresh functionality
- Pull-to-refresh support
- Local cache support
- Cache information
- Detailed post view
- Light and dark theme support
- Animated UI elements

The post cards present the information in a clean and readable format, including the title, description and user information.

The responsive layout allows the application to adapt to different screen sizes.

9. ERROR HANDLING

The application includes different levels of error handling.

API Failure:

If the server returns an unsuccessful response or the request times out, the application handles the failure gracefully.

Invalid JSON:

The application verifies that the API response has the expected structure before converting it into Post objects.

Empty Response:

If the API returns no usable posts, the application handles it as an empty/invalid data state.

No Cache:

If both the API and local cache are unavailable, an appropriate error state is displayed instead of crashing the application.

10. WHAT I LEARNED

Through this assignment, I learned how real Flutter applications communicate with external services.

REST API Integration:

I learned how to make HTTP requests from Flutter and handle responses received from a REST API.

JSON Parsing:

I learned how JSON data received from an API can be converted into structured Dart models.

Async Programming:

I improved my understanding of Future, async and await while working with asynchronous network operations.

Local Storage:

I learned how SharedPreferences can be used to store data locally between application sessions.

Caching:

I understood how caching can improve application reliability and allow previously retrieved data to remain available when the network is unavailable.

Error Handling:

I learned how to handle API failures, invalid responses, empty data and missing cache without crashing the application.

Application Architecture:

Separating the model, API service, cache service and UI helped me understand how to organize Flutter applications into maintainable components.

Responsive UI:

I also practiced creating layouts that adapt to desktop, tablet and mobile screen sizes.

11. CONCLUSION

The PulseAPI project successfully demonstrates REST API integration and local data caching in Flutter.

The application can retrieve data from a REST API, parse JSON responses into Dart models, display the information through a modern responsive dashboard, store successful API responses locally and use cached data when the network is unavailable.

This assignment provided practical experience with:

- REST APIs
- HTTP requests
- JSON parsing
- Future and async/await
- SharedPreferences
- Local caching
- Error handling
- Responsive UI design
- Flutter application architecture

Overall, this assignment helped me move from building static Flutter interfaces toward developing applications that work with external data and real-world network conditions.